

Introdução a Algoritmos

Aula 8 – Ponteiros

Raoni F. S. Teixeira · 1s/2026

Objetivos desta aula

Ao final desta aula, você deverá ser capaz de:

1. Distinguir os três significados do símbolo `*` em C (declaração de ponteiro, desreferência e multiplicação) a partir do contexto;
2. Escrever funções que modificam variáveis do chamador usando passagem por referência com ponteiros;
3. Explicar por que o operador `&` é necessário no `scanf` e o que aconteceria sem ele;
4. Identificar e corrigir os dois erros mais comuns com ponteiros: ponteiro não inicializado e uso após liberar.

1 O problema em aberto

Na Aula 7, terminamos com um experimento inconclusivo: a função `troca` não conseguia modificar as variáveis de `main` porque a passagem por valor só entrega **cópias**. Prometemos que a solução viria nesta aula.

A solução é **ponteiros**. Mas antes de qualquer sintaxe, precisamos construir o modelo mental correto — porque ponteiros são uma das fontes mais comuns de confusão em C, e quase sempre essa confusão vem de não entender o que está acontecendo na memória.

2 A memória como array de bytes

A memória do computador pode ser imaginada como uma sequência muito longa de caixinhas numeradas que armazenam exatamente um byte. O número de cada caixinha é o seu **endereço**.

Endereço	1000	1001	1002	1003	1004	1005	1006	1007
Conteúdo	10	0	0	0	20	0	0	0

Bytes 1000-1003: `int x = 10` Bytes 1004-1007: `int y = 20`

Quando você declara `int x = 10`, o compilador reserva geralmente 4 bytes contíguos na memória (pois `int` tem 32 bits = 4 bytes numa máquina moderna) e armazena o valor 10 nesses bytes. O **endereço** da variável `x` é o número do primeiro byte que ela ocupa — neste exemplo, 1000.

Dois fatos importantes decorrem disso:

- **Toda variável tem um endereço.** Não importa o tipo — int, float, char — toda variável ocupa uma região da memória e tem um endereço inicial.
- **O endereço é um número.** É um inteiro sem sinal que identifica uma posição na memória. Em computadores modernos de 64 bits, esse número tem 8 bytes.

3 O operador &: “me dá o endereço de”

O operador & (lê-se “endereço de”) aplicado a uma variável produz o endereço de memória onde ela está armazenada.

```
#include <stdio.h>

int main() {
    int x = 10;
    int y = 20;

    printf("Valor de x:      %d\n", x);
    printf("Endereço de x:   %p\n", &x);
    printf("Valor de y:      %d\n", y);
    printf("Endereço de y:   %p\n", &y);
    return 0;
}
```

Uma saída típica (os endereços variam a cada execução):

```
Valor de x:      10
Endereço de x:   0xABC0
Valor de y:      20
Endereço de y:   0xABC4
```

O especificador %p imprime um endereço em hexadecimal. Observe que y está 4 bytes após x (0xABC4 - 0xABC0 = 4) — exatamente o tamanho de um int.

Você usa & desde a Aula 2, em todo scanf. Agora o significado é claro: scanf precisa do **endereço** de n — escrito como &n — para poder armazenar o valor lido diretamente na memória de n. Se você passasse apenas n (o valor), scanf não saberia **onde** escrever. O & que parecia “mágica” na Aula 2 é simplesmente o operador de endereço.

4 Variáveis do tipo ponteiro

Se endereços são números, podemos armazená-los em variáveis — variáveis cujo conteúdo é um endereço de memória. Essas variáveis chamam-se **ponteiros**.

Definição — Pontoeiro

Um ponteiro é uma variável que armazena o **endereço de memória** de outra variável. Diz-se que o ponteiro **aponta para** a variável cujo endereço contém.

Isso muda a natureza da peça “dados” da Aula 1. Até aqui, o dado era o valor em si — um número, um vetor de números, um registro de campos. A partir de agora, o dado pode ser a **localização** de outro dado na memória.

A declaração de um ponteiro usa o símbolo `*` entre o tipo e o nome:

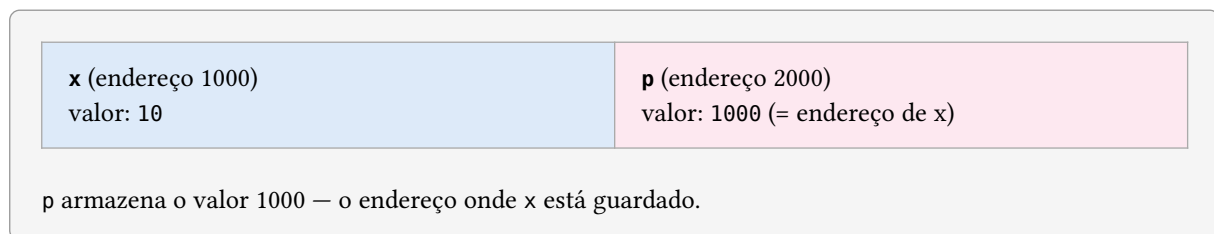
```
int *p;      /* p é um ponteiro para int */
float *q;    /* q é um ponteiro para float */
char *r;     /* r é um ponteiro para char */
```

O tipo antes do `*` indica o tipo da variável **para a qual o ponteiro aponta** — não o tipo do ponteiro em si (todos os ponteiros têm o mesmo tamanho, pois todos armazenam um endereço).

Para fazer um ponteiro apontar para uma variável, atribuímos a ele o endereço dessa variável:

```
int x = 10;
int *p = &x; /* p contém o endereço de x -- p 'aponta para' x */
```

Visualizando na memória:



5 O operador `*`: acessar o valor num endereço

Ter o endereço de uma variável só é útil se podemos usá-lo para acessar ou modificar o valor armazenado nesse endereço. O operador `*` (chamado de **desreferência** ou **indireção**) faz exatamente isso: dado um ponteiro, ele acessa o valor na posição de memória para a qual o ponteiro aponta.

```
int x = 10;
int *p = &x;

printf("%d\n", *p); /* imprime 10 — o valor de x, via ponteiro */

*p = 42;           /* modifica x através do ponteiro */
printf("%d\n", x); /* imprime 42 */
```

A expressão `*p` pode ser lida como “o valor no endereço guardado em `p`” — ou simplesmente “o que `p` aponta”. Quando `*p` aparece no lado esquerdo de uma atribuição, estamos **escrevendo** nesse endereço; quando aparece no lado direito, estamos **lendo**.

5.1 Os três significados de `*` em C

O símbolo `*` aparece em três contextos distintos em C, com significados completamente diferentes:

Contexto	Exemplo	Significado
Declaração	<code>int *p;</code>	“ <code>p</code> é um ponteiro para <code>int</code> ” — define o tipo da variável
Desreferência	<code>*p = 42;</code>	“acesse o valor no endereço guardado em <code>p</code> ” — operador unário

Multiplicação	<code>a * b</code>	“multiplique a por b” — operador binário aritmético
---------------	--------------------	---

O contexto (declaração, expressão com um operando, expressão com dois operandos) sempre deixa claro qual dos três significados está em uso. Quando a distinção parecer ambígua durante a leitura, pare e identifique o contexto explicitamente.

6 Passagem por referência: resolvendo a troca

Agora temos tudo que precisamos para resolver o problema da Aula 7. Em vez de passar os **valores** de `x` e `y` para a função `troca`, passamos os seus **endereços**. A função recebe ponteiros e usa a desreferência para modificar os valores originais diretamente.

```
#include <stdio.h>

void troca(int *a, int *b) {    /* a e b são ponteiros para int */
    int temp = *a;             /* temp recebe o valor no endereço a */
    *a = *b;                   /* escreve no endereço a o valor do endereço b */
    *b = temp;                 /* escreve no endereço b o valor de temp */
}

int main() {
    int x = 10, y = 20;
    printf("Antes: x=%d, y=%d\n", x, y);

    troca(&x, &y);              /* passa os endereços, não os valores */

    printf("Depois: x=%d, y=%d\n", x, y);
    return 0;
}
```

Saída:

Antes: x=10, y=20

Depois: x=20, y=10

Desta vez a troca funciona. Vamos rastrear o que acontece:

Mundo de main

`x` no endereço 1000 → valor 10

`y` no endereço 1004 → valor 20

Mundo de troca

`a` → contém 1000 (endereço de `x`)

`b` → contém 1004 (endereço de `y`)

`*a = *b` escreve 20 **em** 1000

`*b = temp` escreve 10 **em** 1004

troca não recebe cópias dos valores — recebe mapas para os originais.

A diferença em relação à versão da Aula 7 é fundamental: antes, `a` e `b` eram **cópias dos valores** de `x` e `y` — modificá-las não afetava `main`. Agora, `a` e `b` são **cópias dos endereços** de `x` e `y` — e através desses endereços, a função escreve diretamente na memória de `main`.

Passagem por referência em C é, portanto, passagem por valor de um endereço. Não há um mecanismo separado — é a mesma regra de sempre, aplicada a um tipo diferente de valor.

7 Funções que “retornam” múltiplos valores

O mecanismo de `return` só permite devolver um único valor. Mas com ponteiros, uma função pode **modificar** quantas variáveis do chamador forem necessárias — simulando múltiplos retornos.

Exemplo — Mínimo e máximo em uma única função

```
#include <stdio.h>

/*
 * Lê n inteiros e armazena o menor em *min e o maior em *max.
 * Pré-cond.: n >= 1
 */
void minmax(int n, int *min, int *max) {
    int x;
    scanf("%d", &x);
    *min = x;
    *max = x;

    for (int i = 1; i < n; i++) {
        scanf("%d", &x);
        if (x < *min) *min = x;
        if (x > *max) *max = x;
    }
}

int main() {
    int n, menor, maior;
    printf("Quantos números? ");
    scanf("%d", &n);

    minmax(n, &menor, &maior);

    printf("Menor: %d\n", menor);
    printf("Maior: %d\n", maior);
    return 0;
}
```

`minmax` não retorna nada (`void`) — mas ao final da chamada, `menor` e `maior` em `main` contêm os resultados corretos, porque a função escreveu diretamente nos seus endereços.

8 Erros clássicos com ponteiros

Ponteiros são poderosos exatamente porque permitem acessar qualquer posição da memória — inclusive posições que não deveriam ser acessadas. Os dois erros a seguir são os mais comuns e os mais perigosos.

8.1 Ponteiro não inicializado

```
int *p;          /* p existe, mas contém um endereço aleatório (lixo) */
*p = 42;         /* escreve 42 em algum lugar aleatório da memória – PERIGO */
```

Um ponteiro declarado mas não inicializado contém o valor que estiver naquele espaço de memória — que pode ser qualquer coisa. Desreferenciar esse ponteiro é acesso a uma posição arbitrária da memória: o programa pode travar imediatamente (segmentation fault), corromper dados silenciosamente, ou aparentemente funcionar — dependendo do que houver naquele endereço.

Atenção

Nunca desreferencie um ponteiro que não foi inicializado com um endereço válido. Sempre inicialize ponteiros no momento da declaração:

```
int x = 10;
int *p = &x;    /* inicializado com endereço válido */
```

Se ainda não há uma variável para apontar, inicialize com NULL:

```
int *p = NULL; /* ponteiro nulo – claramente "não aponta para nada" */
```

8.2 Ponteiro nulo

NULL é uma constante que representa um ponteiro que não aponta para nenhum endereço válido. É o equivalente do “não há valor” para ponteiros. Desreferenciar um ponteiro nulo causa sempre uma falha imediata do programa (**segmentation fault** em sistemas Unix, acesso a endereço inválido no Windows) — o que, embora pareça ruim, é na verdade melhor do que um ponteiro com lixo: a falha é imediata e o local do erro é fácil de identificar.

```
int *p = NULL;
*p = 42;    /* segmentation fault – falha imediata e identificável */
```

Antes de desreferenciar um ponteiro que pode ser nulo, verifique:

```
if (p != NULL) {
    *p = 42;    /* seguro */
}
```

8.3 Resumo dos erros

Situação	O que acontece	Como evitar
Ponteiro não inicializado	Comportamento indefinido: pode travar, corromper dados ou nada	Sempre inicializar com &variavel ou NULL
Ponteiro nulo desreferenciado	Falha imediata e identificável (segfault)	Verificar p != NULL antes de desreferenciar
Ponteiro para variável destruída	Comportamento indefinido: a variável saiu do escopo mas o endereço foi guardado	Nunca retornar o endereço de uma variável local

O terceiro erro da tabela merece atenção especial:

```
int *ponteiro_perigoso() {
    int x = 10;
    return &x;    /* ERRADO: x será destruída ao retornar */
}
```

Quando a função termina, a variável local `x` deixa de existir — mas o endereço foi devolvido ao chamador, que agora aponta para uma região de memória que pode ser reutilizada por qualquer outra coisa. Usar esse ponteiro é comportamento indefinido.

9 Exemplo integrador: duas funções com ponteiros

O programa abaixo lê uma sequência de inteiros, calcula a média e ao mesmo tempo encontra o desvio máximo em relação à média — tudo usando funções que comunicam múltiplos resultados via ponteiros.

```
#include <stdio.h>

/*
 * Lê n inteiros, calcula sua soma e o maior valor.
 * Resultados escritos em *soma e *maior.
 */
void le_e_analisa(int n, float *soma, int *maior) {
    int x;
    scanf("%d", &x);
    *soma = x;
    *maior = x;

    for (int i = 1; i < n; i++) {
        scanf("%d", &x);
        *soma += x;
        if (x > *maior) *maior = x;
    }
}

/*
 * Retorna o desvio absoluto entre valor e media.
 */
float desvio(int valor, float media) {
    float d = valor - media;
    if (d < 0) d = -d; /* valor absoluto sem usar abs() */
    return d;
}

int main() {
    int n, maior;
    float soma, media;

    printf("Quantos números? ");
    scanf("%d", &n);

    le_e_analisa(n, &soma, &maior);
    media = soma / n;

    printf("Média: %.2f\n", media);
    printf("Maior: %d\n", maior);
    printf("Desvio do maior em relação à média: %.2f\n",
        desvio(maior, media));

    return 0;
}
```

10 Ponteiros e a próxima aula

Ponteiros parecem complexos quando vistos isoladamente. Mas eles são apenas uma ferramenta para responder uma pergunta simples: **onde** na memória está um determinado dado?

Essa pergunta torna-se ainda mais central na próxima aula, sobre vetores. Um vetor em C é essencialmente um bloco contíguo de memória — e o nome de um vetor é, na prática, um ponteiro para o seu primeiro elemento. Tudo que aprendemos sobre ponteiros hoje será usado diretamente para entender como vetores funcionam e como passá-los para funções.

11 Exercícios

1. O que imprime o programa abaixo? Faça o teste de mesa antes de executar.

```
int x = 5, y = 3;
int *p = &x, *q = &y;
*p = *p + *q;
*q = *p - *q;
*p = *p - *q;
printf("%d %d\n", x, y);
```

O que essa sequência de operações faz?

2. Escreva uma função void `divide(int a, int b, int *quociente, int *resto)` que calcula o quociente e o resto da divisão inteira de a por b e os armazena nos endereços fornecidos. Teste com a = 17, b = 5.
3. Escreva uma função void `ordena2(int *a, int *b)` que garante que após a chamada `*a <= *b`. Use a função `troca` desenvolvida nesta aula.
4. Explique com suas palavras a diferença entre:
 - `int x = 10;` e `int *p = &x;`
 - `p` e `*p`
 - `&x` e `x`
5. (*Desafio*) Escreva uma função void `minmax_func(int n, int *min, int *max, float *media)` que lê n inteiros e calcula simultaneamente o mínimo, o máximo e a média, escrevendo os três resultados nos endereços fornecidos. Escreva `main` de forma que fique com apenas quatro linhas de lógica.